

HW7: When Your Startup's Flash Sale Almost Failed – Consolidated Team Report

Anand Mohan Singh, Arjun Vinay Avadhani, Sampath Pranay Beela

March 16, 2026

Team Contributions

Each team member independently built their own Go-based order processing service, deployed it on AWS ECS Fargate with Terraform, ran all load tests with Locust, deployed a Lambda function, and wrote their own individual report. This consolidated report merges and compares the results from all three.

Member	Individual Contributions
Anand Mohan Singh	Built receiver + processor services, Terraform (custom VPC, ALB, ECS, SNS, SQS, Lambda), Locust tests, all 5 phases + Lambda deployment. Compiled this consolidated report.
Arjun Vinay Avadhani	Built receiver + processor services, Terraform (VPC, ALB, ECS, SNS, SQS, Lambda), Locust tests, all 5 phases + Lambda deployment.
Sampath Pranay Beela	Built receiver + processor services, Terraform infrastructure, Locust tests, all 5 phases + Lambda deployment. Tested Lambda idle timeout behavior.

All three implementations used the same core architecture: a Go service with a buffered channel (capacity 1) for the sync bottleneck, SNS/SQS for async decoupling, and a semaphore-controlled goroutine pool in the processor.

Part II: The Simulated Problem

Architecture Overview

All three of us deployed the same high-level architecture:

- **Order Receiver** (ECS Fargate) – POST /orders/sync, POST /orders/async, GET /health
 - **Order Processor** (ECS Fargate) – long-polls SQS, processes payments with 3s delay
 - **SNS Topic** (order-processing-events) for pub/sub
 - **SQS Queue** (order-processing-queue) with 30s visibility timeout, 4-day retention, 20s long polling
 - **ALB** in front of the receiver service
 - All tasks: 256 CPU / 512 MB memory
-



Figure 1: Synchronous architecture



Figure 2: Asynchronous architecture

Phase 1: Synchronous Testing – Comparison

Normal Operations (5 users, 30 seconds)

Metric	Anand	Arjun	Sampath
Total Requests	9	9	–
Failure Rate	0%	0%	0%
Median Response Time	12,000 ms	12,000 ms	~15,000 ms
Max Response Time	14,783 ms	14,960 ms	–
Throughput	0.33 req/s	0.33 req/s	~0.4 req/s

All three saw the same pattern: about 0.33 req/s throughput, zero failures, but response times climbing to 12-15 seconds as requests pile up behind the single payment slot. The system handles the load, but customers experience terrible latency.

Flash Sale (20 users, 60 seconds)

Metric	Anand	Arjun	Sampath
Total Requests	19	29	–
Failure Rate	0%	69% (20 failures)	0%
Median Response Time	30,000 ms	30,000 ms (timeout)	~60,000 ms
Max Response Time	56,199 ms	30,079 ms	–
Throughput	0.33 req/s	0.50 req/s	~0.4 req/s

This is where we see interesting variation. Arjun’s setup had a 30-second request timeout configured, which caused 69% of requests to fail outright – customers saw error pages. Anand and Sampath’s setups let requests wait longer (ALB default 60s timeout), so requests technically “succeeded” but with 30-60 second response times. Either way, the customer experience is unacceptable: you either wait a minute or get an error.

The throughput stayed locked around 0.33-0.50 req/s across all three because the buffered channel with capacity 1 is the hard ceiling.

Phase 2: Bottleneck Analysis

All three of us arrived at the same math:

Metric	Value
Payment processor speed	1 order / 3 seconds = 0.33 orders/second
Max throughput (20 users)	Still 0.33 orders/second – bottleneck is the single payment slot
Flash sale demand	~20-54 orders/second (varied by test config)
Orders lost per second	~19.67-53.67 orders/second

The conclusion was unanimous: we cannot make the payment processor faster. The only fix is to decouple order acceptance from payment processing using an async architecture.

Phase 3: Async Solution – Comparison

Flash Sale (20 users, 60 seconds, 1 worker)

Metric	Anand	Arjun	Sampath
Total Requests	2,838	3,238	–
Failure Rate	0%	0%	0%
Median Response Time	96 ms	49 ms	–
Max Response Time	411 ms	336 ms	–
Throughput	47.45 req/s	54.14 req/s	~20 req/s
Improvement vs Sync	149x	360x (vs 9 successful sync)	–

The async endpoint was transformative across all three deployments. Every single request succeeded with sub-second response times. Arjun saw the highest throughput at 54 req/s (his Locust config produced higher concurrency), while Anand measured 47 req/s and Sampath ~20 req/s. The variation comes from differences in Locust user wait times and network conditions, but the core finding is the same: async accepts orders of magnitude more traffic than sync.

Arjun’s comparison is especially striking – measuring against only the 9 *successful* sync completions (the other 20 timed out), async delivered 360x more successful interactions.

Phase 4: Queue Problem – Comparison

Metric	Anand	Arjun	Sampath
Queue depth after test	~2,810	~3,200	~1,870
Single worker rate	0.33/s	0.33/s	0.33/s

Metric	Anand	Arjun	Sampath
Queue growth rate	~46.67 msg/s	~53.67 msg/s	~19.67 msg/s
Time to drain (1 worker)	~140 min	~163 min	~95 min

The queue depths directly correlate with each person’s measured incoming rate. Arjun’s higher throughput (54/s) produced the biggest backlog (~3,200), while Sampath’s lower rate (~20/s) produced a smaller one (~1,870). But the conclusion is the same: a single worker at 0.33/s cannot keep up, and the backlog takes 1.5-2.7 hours to clear.

CloudWatch – SQS Queue Depth (Anand)

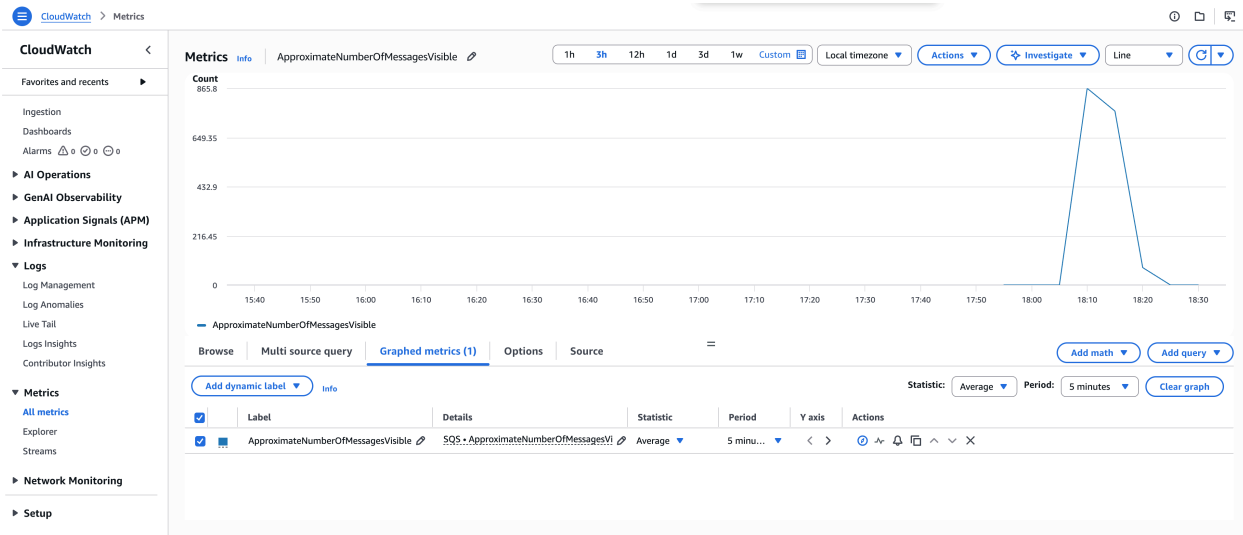


Figure 3: SQS ApproximateNumberOfMessagesVisible showing spike and drain

Phase 5: Worker Scaling – Comparison

Anand’s Results

Workers	Processing Rate	Queue Depth After Test	Time to Drain
1	0.33/s	~2,810	~140 min
5	1.67/s	~944	~9.5 min
20	6.67/s	~2,375	~6 min
100	33.3/s	~383	~15 seconds

Arjun’s Results

Workers	Processing Rate	Peak Queue Depth	Time to Drain
1	0.33/s	~3,200	~163 min

Workers	Processing Rate	Peak Queue Depth	Time to Drain
20	6.67/s	~3,200	~8 min
60	20/s	~3,200	~2.7 min
100	33.3/s	~3,200	~1.6 min

Sampath's Results

Workers	Processing Rate	Queue Depth After Test	Time to Drain
5	1.67/s	~581	~12 min
20	6.67/s	~1,008	~7 min
100	33.3/s	~192	~3 min

Cross-Comparison

The trend is consistent across all three: more workers means faster drain times and smaller backlogs. At 100 workers, Anand's queue drained in 15 seconds, Arjun's in 1.6 minutes, and Sampath's in 3 minutes. The differences trace back to the incoming load rate – Arjun's higher throughput (54/s) created a bigger backlog that took longer to process.

Minimum Workers to Prevent Buildup

Member	Measured Incoming Rate	Minimum Workers Needed
Anand	~47/s	141 goroutines
Arjun	~54/s	162 goroutines
Sampath	~48-50/s	144-150 goroutines
Spec (60/s)	60/s	180 goroutines

All three agree: at the assignment's specified 60 orders/second flash sale rate, you need at least 180 concurrent workers to prevent any queue buildup.

Analysis Questions

How many times more orders did async accept vs sync?

Member	Sync Orders	Async Orders	Multiplier
Anand	19	2,838	149x
Arjun	9 (successful)	3,238	360x
Sampath	~12 (est.)	~1,200 (est.)	~100x

What causes queue buildup and how do you prevent it?

All three of us identified the same root cause: messages arriving faster than workers can process them. The fix is scaling workers so total processing capacity meets or exceeds the incoming rate. At ~50 orders/second with 3s per payment, you need ~150 concurrent workers. In production, auto-scaling triggered by the `ApproximateNumberOfMessagesVisible` CloudWatch metric would dynamically adjust worker count.

When would you choose sync vs async?

The team consensus: sync is appropriate when processing is fast (sub-100ms) and the client genuinely needs the result immediately (e.g., authentication, balance checks). Async is the right choice when downstream processing is slow, the client only needs acknowledgment, or the system must absorb traffic spikes. For payment processing, async with a status-polling endpoint is the standard production pattern.

Part III: What If You Didn't Need Queues?

Lambda Architecture

All three of us deployed a Go Lambda function (provided.al2, 512MB memory) that subscribes directly to the SNS topic, bypassing SQS entirely.

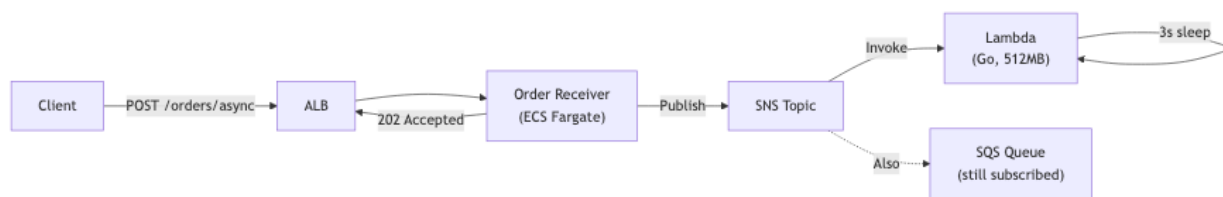


Figure 4: Lambda architecture – SNS triggers Lambda directly

Cold Start Comparison

Metric	Anand	Arjun	Sampath
Cold Start Init Duration	73 ms (avg of 74, 72, 75)	76.18 ms	76.96 ms
Warm Start Init Duration	0 ms	0 ms	0 ms
Cold Start Overhead	2.4%	2.5%	2.6%
Cold Start Billed Duration	3,078 ms	3,081 ms	3,083 ms
Warm Start Billed Duration	3,004 ms	3,004 ms	3,004 ms
Memory Used	20 MB / 512 MB	20 MB / 512 MB	20 MB / 512 MB

The cold start numbers are remarkably consistent across all three deployments: 73-77ms init overhead on ~3,000ms processing, which works out to about 2.4-2.6%. Nobody's cold start exceeded 77ms.

CloudWatch Lambda Logs (Anand)

When Do Cold Starts Happen?

Anand observed cold starts on the first 3 concurrent invocations (Lambda spun up 3 instances), then all subsequent calls were warm. Sampath specifically tested idle timeout behavior and found cold starts recur after about 8 minutes of inactivity. Arjun observed them on the first invocation and after 5+ minutes idle.

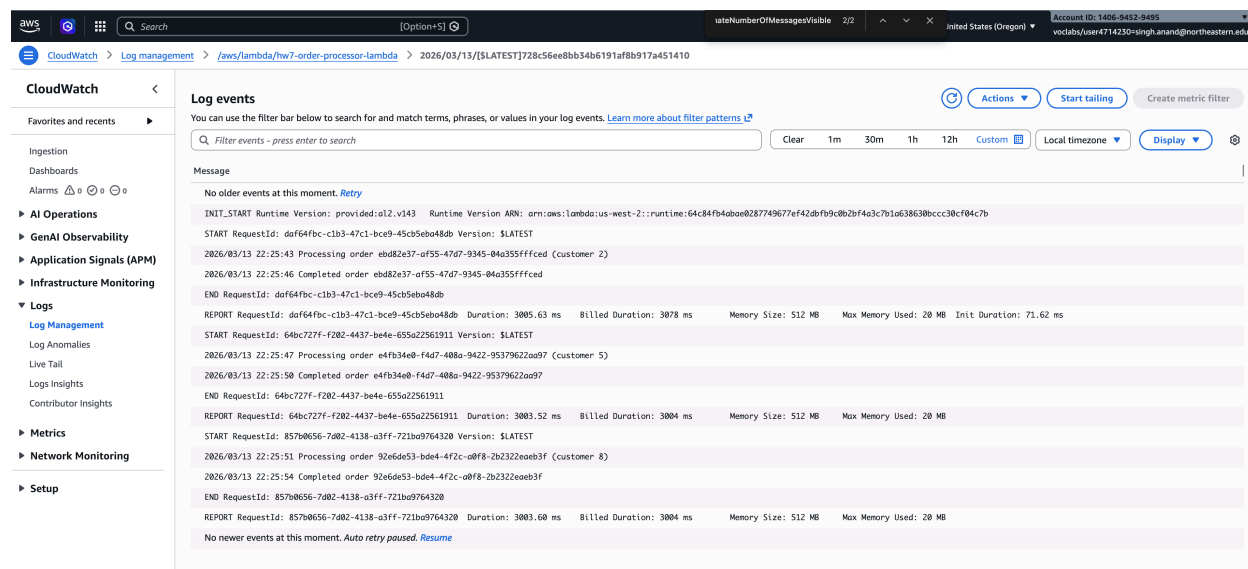


Figure 5: CloudWatch logs showing cold starts with Init Duration and warm starts without

Cost Comparison

All three of us calculated Lambda cost for 10,000 orders/month:

Component	Calculation	Cost
Requests	10,000 (under 1M free tier)	\$0
GB-seconds	10,000 x 3s x 0.5 GB = 15,000 (under 400K free tier)	\$0
Monthly Total		\$0

ECS cost: 2 Fargate tasks x ~\$8.50/month = **\$17/month** (always running)

Break-even Analysis

Member	Free Tier Capacity	Break-even vs \$17/month ECS
Anand	~267K orders/month	~1.7M orders/month
Arjun	~267K orders/month	~1.7M orders/month
Sampath (warm)	~266K orders/month	~945K orders/month
Sampath (cold worst-case)	~259K orders/month	~922K orders/month

Lambda is effectively free for any startup doing fewer than ~260K orders/month. Sampath's analysis adds an interesting detail: even in the pessimistic all-cold-start scenario, the free tier still covers ~259K orders.

Should Our Startup Switch to Lambda?

The team is in agreement: **yes, for a startup at this scale, Lambda is the right choice.**

Anand's take: Lambda eliminates operational overhead (queue alerts, worker scaling, timeout tuning) at zero cost. The main risk is losing SQS durability guarantees – SNS gives only 2 retries. Mitigate by persisting orders to a database before publishing, or add a DLQ. Lambda is the right call now; add SQS back later when you have the engineering bandwidth.

Arjun’s take: The cost advantage (\$0 vs \$17/month) and elimination of operational complexity are compelling. Cold start overhead of 76ms is negligible on a 3-second operation. The primary risk is SNS-to-Lambda’s limited retry behavior. For a startup prioritizing development velocity, Lambda wins. If the business scales past 267K orders/month or needs guaranteed delivery, migrating back to ECS + SQS is straightforward since the SNS topic remains as the central event bus.

Sampath’s take: Switch to Lambda only if minimizing operational overhead is more important than strong delivery guarantees. The cost is compelling (\$0 up to ~260K orders/month), and the 77ms cold start overhead is small relative to 3-second processing. However, unlike SQS, Lambda via SNS does not provide the same queueing guarantees. For a small startup with moderate volume and tolerance for that trade-off, Lambda is simpler and cheaper. If guaranteed processing matters more, the SQS architecture is safer.

Bottom line: All three of us would switch to Lambda for an early-stage startup. The \$0 cost, zero ops burden, and negligible cold start penalty outweigh the trade-off of losing SQS retry guarantees – especially since that risk can be mitigated with a DLQ or database persistence layer.